

Drift Detector + Streamlit Dashboard

1. Mahalanobis distance from a centroid (drift detector core)

Plan: nightly job computes a baseline centroid per route; sessions >3 Mahalanobis distance from it get flagged.

In plain terms: Euclidean distance treats every direction as equally "normal." Mahalanobis stretches that into an oval shaped by how the data actually varies, so it judges "unusual" relative to real spread, not a flat circle.

Where it can break:

- Needs a stable covariance matrix — with 768 dimensions, that requires well more samples per route than 768, or the matrix goes unstable/non-invertible. Worth checking per-route sample counts before trusting this.
- Assumes "normal" is roughly one Gaussian blob. Our normal cohort spans Q&A, coding, and summarization — genuinely different topics. A single centroid can sit in a "middle" that's close to none of them, false-flagging legitimate but topically different traffic.

Alternatives:

- **Simple centroid + cosine/Euclidean distance** — simpler, no covariance risk, less principled. Fallback if Mahalanobis proves fragile.
- **Isolation Forest / One-Class SVM** — no shape assumption, but far less explainable in a live demo than "3 from baseline." Wouldn't switch for this week.
- **Cluster-aware baseline (the one I'd pitch):** cluster normal traffic into a few sub-clusters first — the same k-means step Page 1 already needs — then score distance to the *nearest* sub-cluster, not one global centroid. Fixes the multi-topic problem, and reuses the dashboard's clustering work rather than duplicating it.

Baseline poisoning risk: if the nightly recompute includes traffic that was itself already flagged, the baseline can drift toward the attack over time. Mitigation: exclude anything already in `drift_events` from the recompute — partial fix, doesn't catch a campaign that stays under threshold the whole time. More below.

2. Streamlit for the dashboard

Why it's the right call: the whole stack is already Python (embeddings, pgvector, k-means) — Streamlit avoids a separate JS frontend, which matters given this is one week, not a frontend sprint.

Alternatives considered: Grafana (great for time series/alerting, bad at UMAP-style scatter plots — the wrong shape of tool for Page 1); Dash/Flask (more control, no real benefit here); Metabase/Superset (fine for tabular data, not cluster visualization).

Where it can break: re-runs the whole script on every interaction, so uncached UMAP would recompute per click — exactly why the plan already calls for precomputing and caching it. Worth persisting that cache to disk, not just `@st.cache_data`, so a restart doesn't silently trigger a slow recompute.

3. UMAP vs t-SNE

The deciding factor is `.transform()`, not speed. UMAP can project new points onto an already-fit projection; t-SNE requires a full recompute for any new data. Since real traffic will keep arriving after this demo, that incremental capability matters more than raw speed.

Other options: PCA (fast, linear, muddier cluster separation — a fallback, not a first choice); newer methods like PaCMAP (aware of them, not worth the risk on a less battle-tested library with a week on the clock).

4. "Nothing looks anomalous" risk

Concrete plan, not just awareness: we have ground-truth cohort labels in the synthetic data. Before demo day, compute Mahalanobis scores for known normal vs drift sessions, plot the distributions, and pick a threshold that actually separates them — instead of trusting 3 as a default. If 3 is too strict or too loose, I'll know the actual numbers ahead of the dry run, not during it.

Open Questions (not decisions for this week, just on record)

Long prompts vs. embedding context limits. Nomic's 8192-token window raises the ceiling over MiniLM's 256, but doesn't remove it — nothing in `embedder.py` currently checks length or chunks text, and what happens today past 8192 tokens (truncation vs. error) hasn't been verified. Standard fix is overlapping-window chunking, but it has a real limit worth naming: some intent is distributed across the whole prompt, so a slow-building attack could have every individual chunk look fine. Worth treating as a known blind spot, not a solved problem.

Baseline poisoning, beyond the exclusion rule above. Excluding flagged events only protects against attacks that already crossed threshold once — a campaign that stays under threshold the whole time still biases the baseline. Broader directions, undecided: a trusted reference dataset instead of a pure rolling window; robust statistical estimators less sensitive to contamination; or multiple semantic baselines instead of one — which is the same mechanism as the cluster-aware baseline idea above, just also making poisoning harder since an attacker would need to bias several baselines at once.

Suggested flow for the conversation

1. Quick honest status: Week 4 pipeline/embeddings are validated (including a real bug found and fixed in retrieval) — establishes the foundation is trustworthy, not just "ran without crashing."
2. Drift detector plan pitch the cluster-aware baseline as the strongest idea, tied to the dashboard's clustering step.
3. Streamlit choice, briefly justified against Grafana.
4. UMAP vs t-SNE, leading with `.transform()`.
5. The ground-truth threshold-tuning plan for the "nothing looks anomalous" risk.
6. Ask directly: is the cluster-aware baseline worth this week's extra time, or ship the simpler centroid now and document the improvement as a next step? Real question, not rhetorical.
7. If time allows, mention the two open questions as known limitations on record — not asking for a decision now.